

Análisis Amortizado y Diseño Circular: The Snake Challenge

Integrantes:

Yoseth Santiago Cardenas Clavijo
Franklin Orlando Delgado Ramirez
Camilo Andres Triana Ortiz

Grupo: Estructuras de Datos 03 - Grupo 5

24 de septiembre de 2026

1. Representación Interna

Para satisfacer los requisitos de `addHead` en $O(1)$ amortizado y `removeTail` en $O(1)$ *worst-case*, la representación física de la serpiente (clase **Snake**) se implementa como un **Arreglo Circular** (Circular Buffer) sobre un arreglo nativo de Java.

Se definen las siguientes variables de estado:

- **data:** Arreglo nativo `Position[]` que almacena los elementos.
- **size:** Entero que representa la cantidad actual de segmentos que conforman la serpiente.
- **capacity:** Entero que indica el tamaño físico del arreglo `data`.
- **tail:** Entero que indica el índice físico en el arreglo `data` donde se encuentra la cola (índice lógico 0).
- **head:** Entero que indica el índice físico en el arreglo `data` donde se encuentra la cabeza (índice lógico $k - 1$).

Relación entre posiciones lógicas y físicas:

Un índice lógico i , donde $0 \leq i < \text{size}$, se transforma en su posición física correspondiente mediante aritmética modular:

$$\text{Posición Física} = (\text{tail} + i) \pmod{\text{capacity}} \quad (1)$$

Esta representación permite que la serpiente “avance” lógicamente por el arreglo sin necesidad de desplazar los elementos físicamente. Si la cabeza o la cola superan el final del arreglo, dan la vuelta hacia la posición 0, haciendo uso eficiente de las posiciones que quedan libres atrás.

2. Invariantes

Para garantizar la correctitud de la estructura y de sus operaciones, se mantienen los siguientes invariantes antes y después de cada operación pública:

1. **Límites de tamaño:** $0 \leq \text{size} \leq \text{capacity}$ y $\text{capacity} \geq 2$ (dado que inicia en 2).
2. **Límites de índices físicos:** $0 \leq \text{tail} < \text{capacity}$ y $0 \leq \text{head} < \text{capacity}$.
3. **Relación entre Punteros:** Si $\text{size} > 0$, la posición de la cabeza está determinada estrictamente por la cola y el tamaño: $\text{head} = (\text{tail} + \text{size} - 1) \pmod{\text{capacity}}$.
4. **Preservación del orden (Resize):** Inmediatamente después de ejecutar `resize`, $\text{tail} = 0$ y los elementos ocupan posiciones físicas consecutivas desde 0 hasta $\text{size} - 1$.

3. Análisis de Complejidad

- **DynamicArray (P1):** Los accesos `get` y `set` acceden directamente al índice del arreglo en memoria, por lo tanto, son $O(1)$ **worst-case**. `removeLast` decrece la variable `size` y elimina la referencia, tomando $O(1)$ **worst-case**. `append` es $O(1)$ **amortizado**; aunque su *worst-case* es $O(n)$ al duplicar el arreglo, los n insertados previamente financian esta única copia.
- **Problema de la implementación ingenua (P2):** La operación propuesta `REMOVE.TAIL()` tiene un bucle `for` que desplaza a la izquierda todos los $k-1$ elementos restantes. Esto hace que eliminar la cola sea $O(k)$ **worst-case**. Para m movimientos sin comer, la complejidad total es $O(m \cdot k)$. Es falso afirmar que `MOVE()` es $O(1)$ amortizado, porque el cuello de botella no es la inserción, sino el desplazamiento físico constante de $O(k)$ elementos cada vez que la serpiente avanza.
- **Rediseño Circular (P3):** En el arreglo circular propuesto, `removeTail()` solo actualiza `tail = (tail + 1) % capacity` y resta 1 a `size`. No hay bucles. Su complejidad es estrictamente $O(1)$ **worst-case**. Por otro lado, `addHead()` incluye la posibilidad de redimensionamiento (arreglo lleno), manteniendo la propiedad de $O(1)$ **amortizado**.

4. Demostración del Análisis Amortizado

Consideremos g manzanas consumidas durante m movimientos. La longitud máxima transitoria de la serpiente será $n = 2$ (inicial) + g (manzanas) + 1 (nueva cabeza antes de borrar cola).

- **Costo total de movimientos:** Los m movimientos implican m llamadas a `addHead` y $m - g$ llamadas a `removeTail` (estas últimas cuestan siempre $O(1)$ absoluto). La parte que depende exclusivamente de los movimientos sin crecimiento (gestión de `head/tail`) cuesta $O(m)$. Las expansiones (`resize`) dependen únicamente del crecimiento (las manzanas g).
- **Demostración (Método Agregado):** El tamaño inicia en $C = 2$. Las copias solo ocurren en capacidades potencia de 2: $2, 4, 8, \dots, 2^k$ donde $2^k \leq n$. El costo total de copias es la serie geométrica $\sum_{i=1}^{\lfloor \log_2 n \rfloor} 2^i = 2^{\lfloor \log_2 n \rfloor + 1} - 2 < 2n$. Ya que el número total de elementos copiados es estrictamente menor a $2n$, el costo total en redimensionamientos es acotado por $O(n) \equiv O(g)$. Por lo tanto, el costo total de toda la ejecución es $O(m + g)$ y el costo amortizado de `addHead` es $O(1)$.
- **Explicación sobre `removeTail` (Potencial):** Al eliminar elementos, `removeTail` reduce el tamaño físico de la serpiente pero el arreglo NO reduce su capacidad. Esto significa que la estructura tardará *más* en llenarse de nuevo, aplazando el siguiente redimensionamiento. Esto garantiza que las operaciones `addHead` que reemplazan espacios vaciados aportarán su “crédito” amortizado sin disparar nuevas copias, asegurando que existan fondos suficientes cuando eventualmente `size = capacity`.

5. Comparación de las Políticas de Crecimiento

Demostración sobre el adversario: En ausencia de una política de contracción (`shrink`), el adversario no puede forzar copias repetidas en el mismo umbral de capacidad. El arreglo solo se expande al alcanzar llenados máximos históricos. Por tanto, sin importar en *qué momento* específico de los m movimientos el adversario decida colocar la manzana, el redimensionamiento ocurrirá en las mismas capacidades $C_1, C_2 \dots$ hasta n . Para lograr copias maliciosas continuas, el adversario requeriría que el sistema redujera su capacidad y la volviera a aumentar intermitentemente.

Familia General de Políticas: Para lograr un costo constante $O(1)$ amortizado es *indispensable* que el crecimiento sea multiplicativo, representado por la familia $C_{new} = \lceil \alpha C \rceil$ donde

$\alpha > 1$. Las progresiones aritméticas como $C + k$ no logran costo constante. El total de copias generadas en la familia exponencial está dada por la serie geométrica que converge asintóticamente a $\frac{\alpha}{\alpha-1}n$.

- Si $\alpha = 2$, la constante es 2 (rápido, pero gasta más memoria).
- Si $\alpha = 1,5$, la constante es 3 (más copias, pero ahorra memoria).

6. Tabla correspondiente a la Pregunta 5

Política	# Resizes	Total copias	Costo amort. por crecimiento	Espacio sin usar tras resize
$C + 1$	$O(n)$	$\frac{n(n-1)}{2} = O(n^2)$	$O(n)$	0 celdas (0 %)
$2C$	$O(\log_2 n)$	$< 2n = O(n)$	$O(1)$	$\frac{C}{2}$ celdas (50 %)
$1,5C$	$O(\log_{1,5} n)$	$< 3n = O(n)$	$O(1)$	$\frac{C}{3}$ celdas ($\approx 33,3$ %)